



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 9

1. ENUNCIADO

Escreva um programa que cria tarefas para realizar N inserções em duas listas encadeadas, cada uma associada a uma thread. Cada thread deve escolher aleatoriamente em qual lista inserir um número. Garanta a integridade das listas evitando condições de corrida e, sempre que possível, use regiões críticas nomeadas para que a inserção em uma lista não bloqueie a outra. Em seguida, generalize o programa para um número de listas definido pelo usuário. Explique por que, nesse caso, regiões críticas nomeadas não são suficientes e por que o uso de locks explícitos se torna necessário.

2. DISCUSSÃO

Nas tarefas anteriores, a paralelização foi feita com `#pragma omp parallel for`, que divide as iterações de um laço entre as threads de forma estática. No entanto, nem sempre o trabalho a ser paralelizado tem essa estrutura regular de laço. Dessa maneira, a OpenMP oferece também o modelo de tarefas, que se mostra mais adequado quando cada unidade de trabalho é independente e pode ser executada por qualquer thread disponível.

2.1. Tarefas em OpenMP (`#pragma omp task`)

A diretiva `#pragma omp task` cria uma unidade de trabalho que é colocada em uma fila e executada por qualquer thread ociosa do time. Geralmente, esse padrão é combinado com `#pragma omp single`, no qual apenas uma thread fica responsável por criar as tarefas, enquanto

as demais ficam disponíveis para executá-las. Assim, ao invés de dividir um laço de forma fixa, o trabalho é distribuído dinamicamente, o que é especialmente útil quando cada inserção é uma operação curta e independente.

2.2. Listas encadeadas

A lista encadeada é uma estrutura de dados na qual cada nó armazena um valor e um ponteiro para o próximo nó. Diferentemente do vetor, ela não ocupa posições contíguas na memória, sendo construída dinamicamente com malloc. Para essa tarefa, foi escolhida a inserção no início da lista, pois ela tem custo constante e não exige percorrer toda a estrutura. Contudo, o ponteiro para a cabeça da lista é uma variável compartilhada: se duas threads inserirem ao mesmo tempo na mesma lista, ambas podem ler o mesmo *cabeca e sobrescrever o ponteiro uma da outra, perdendo nós e corrompendo a estrutura. Esse é justamente o problema da condição de corrida (race condition).

2.3. Regiões críticas nomeadas

Uma forma de proteger a inserção é a diretiva `#pragma omp critical`, que garante que apenas uma thread execute o bloco por vez. No entanto, por padrão, todas as regiões críticas sem nome compartilham o mesmo cadeado global. Isso significa que, mesmo inserindo em listas diferentes, uma thread bloquearia a outra desnecessariamente.

Para resolver isso, a OpenMP permite nomear a região crítica: `#pragma omp critical(nome)`. Dessa forma, regiões com nomes diferentes são independentes entre si, isto é, possuem cadeados distintos. Assim, ao usar `critical(listaA)` e `critical(listaB)`, uma inserção na lista A não bloqueia uma inserção simultânea na lista B, mantendo o paralelismo entre as listas e protegendo cada uma individualmente.

2.4. Locks explícitos (omp_lock_t)

O lock explícito é um cadeado manipulado diretamente pelo programador por meio das funções `omp_init_lock`, `omp_set_lock`, `omp_unset_lock` e `omp_destroy_lock`. Diferentemente da região crítica nomeada, o lock é um

objeto criado em tempo de execução, o que significa que pode ser armazenado em um vetor e indexado dinamicamente. Por causa disso, ele se torna indispensável quando o número de regiões a serem protegidas só é conhecido durante a execução do programa.

2.5. Por que regiões críticas nomeadas não bastam para N listas

Aqui está o ponto central da segunda parte da tarefa. O nome de uma região crítica é um identificador fixo, definido em tempo de compilação, ou seja, não é possível gerar algo como `critical(lista_i)` variando `i` em tempo de execução. Por consequência, quando o número de listas é definido pelo usuário, não há como escrever uma região crítica nomeada para cada lista, já que elas nem existem quando o código é compilado. Restariam apenas duas saídas ruins: usar uma única região crítica sem nome (o que serializaria todas as inserções e destruiria o paralelismo entre listas) ou tentar criar nomes dinâmicos (o que é impossível). Portanto, faz-se necessário o uso de locks explícitos: cria-se um vetor `omp_lock_t locks[num_listas]`, e cada thread trava apenas o lock correspondente à lista que escolheu, mantendo a proteção fina e independente para qualquer quantidade de listas.

2.6. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.7. OUTPUT

Segue no apêndice.

2.8. COMPILAR

```
gcc -fopenmp tarefa9.c -o tarefa9
```

3. CONCLUSÃO

Os resultados confirmam que, em ambos os programas, a integridade das listas foi preservada, pois o total de elementos inseridos coincidiu exatamente com o `N` esperado (1.000.000), e a distribuição entre as listas ficou equilibrada, evidenciando a ausência de condições de corrida.

No Programa 1, as regiões críticas nomeadas se mostraram suficientes e eficientes, já que `critical(listaA)` e `critical(listaB)` funcionam como cadeados independentes,

permitindo que inserções em listas diferentes ocorram simultaneamente, sem bloqueio mútuo. Contudo, essa abordagem só é viável porque o número de listas é fixo e conhecido em tempo de compilação.

Ao generalizar para um número de listas definido pelo usuário, as regiões críticas nomeadas deixam de ser aplicáveis, uma vez que seus nomes são identificadores estáticos e não podem ser gerados dinamicamente em tempo de execução. Por essa razão, o uso de locks explícitos se torna necessário: por serem objetos criados em tempo de execução, eles podem ser armazenados em um vetor e indexados conforme a lista escolhida, garantindo a mesma proteção fina e independente para qualquer quantidade de listas. Portanto, conclui-se que a escolha do mecanismo de sincronização depende diretamente de o número de recursos a proteger ser estático ou dinâmico.

4. APÊNDICE

4.1. RESOLUÇÃO DO PROGRAMA 1

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define N 1000000 // numero de insercoes

// Estrutura do no da lista encadeada
typedef struct No {
    int valor;
    struct No *proximo;
} No;

// Gerador de numeros aleatorios proprio (pro win)
// pois cada tarefa usa sua propria seed)
int meu_rand(unsigned int *seed) {
    *seed = *seed * 1103515245 + 12345;
    return (*seed >> 16) & 0x7fff;
}

// Inserir no inicio da lista (pra custo constante)
void inserir(No **cabeca, int valor) {
    No *novo = (No *) malloc(sizeof(No));
    novo->valor = valor;
    novo->proximo = *cabeca;
    *cabeca = novo;
}
```

```

// Contar quantos elementos a lista possui
int contar(No *cabeca) {
    int total = 0;
    while (cabeca != NULL) {
        total++;
        cabeca = cabeca->proximo;
    }
    return total;
}

// Liberar a memoria da lista
void liberar(No *cabeca) {
    while (cabeca != NULL) {
        No *temp = cabeca;
        cabeca = cabeca->proximo;
        free(temp);
    }
}

int main() {
    No *listaA = NULL;
    No *listaB = NULL;

    double inicio = omp_get_wtime();

    #pragma omp parallel
    {
        // Apenas uma thread cria as tarefas, as outras executam
        #pragma omp single
        {
            for (int i = 0; i < N; i++) {
                #pragma omp task firstprivate(i)
                {
                    // seed propria de cada tarefa
                    unsigned int seed = i + omp_get_thread_num() * 7919;
                    int escolha = meu_rand(&seed) % 2; // 0 ou 1
                    int valor = meu_rand(&seed);

                    if (escolha == 0) {
                        // cadeado exclusivo da lista A
                        #pragma omp critical(listaA)
                        {
                            inserir(&listaA, valor);
                        }
                    } else {
                        // cadeado exclusivo da list B
                        #pragma omp critical(listaB)
                        {

```

```

        inserir(&listaB, valor);
    }
}

}

}

}

double fim = omp_get_wtime();

int qtdA = contar(listaA);
int qtdB = contar(listaB);

printf("Lista A: %d elementos\n", qtdA);
printf("Lista B: %d elementos\n", qtdB);
printf("Total inserido: %d (esperado: %d)\n", qtdA + qtdB, N);
printf("Tempo: %f segundos\n", fim - inicio);

liberar(listaA);
liberar(listaB);

return 0;
}

```

4.2. RESOLUÇÃO DO PROGRAMA 2

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define N 1000000 // n de insercoes

typedef struct No {
    int valor;
    struct No *proximo;
} No;

// Gerador de numeros aleatorios proprio (pro win)
int meu_rand(unsigned int *seed) {
    *seed = *seed * 1103515245 + 12345;
    return (*seed >> 16) & 0x7fff;
}

void inserir(No **cabeca, int valor) {

```

```

    No *novo = (No *) malloc(sizeof(No));
    novo->valor = valor;
    novo->proximo = *cabeca;
    *cabeca = novo;
}

int contar(No *cabeca) {
    int total = 0;
    while (cabeca != NULL) {
        total++;
        cabeca = cabeca->proximo;
    }
    return total;
}

void liberar(No *cabeca) {
    while (cabeca != NULL) {
        No *temp = cabeca;
        cabeca = cabeca->proximo;
        free(temp);
    }
}

int main() {
    int num_listas = 8; // deixar definido em 8

    // Vetor de listas (um ponteiro de cabeca por lista)
    No **listas = (No **) malloc(num_listas * sizeof(No *));
    // Vetor de locks: um cadeado por lista, criado em tempo de execucao
    omp_lock_t *locks = (omp_lock_t *) malloc(num_listas *
sizeof(omp_lock_t));

    for (int i = 0; i < num_listas; i++) {
        listas[i] = NULL;
        omp_init_lock(&locks[i]);
    }

    double inicio = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        {
            for (int i = 0; i < N; i++) {
                #pragma omp task firstprivate(i)
                {
                    unsigned int seed = i + omp_get_thread_num() * 7919;
                    int escolha = meu_rand(&seed) % num_listas;

```

```

        int valor = meu_rand(&seed);

        // trava apenas o lock da lista escolhida
        omp_set_lock(&locks[escolha]);
        inserir(&listas[escolha], valor);
        omp_unset_lock(&locks[escolha]);
    }
}
}

double fim = omp_get_wtime();

int total = 0;
for (int i = 0; i < num_listas; i++) {
    int qtd = contar(listas[i]);
    printf("Lista %d: %d elementos\n", i, qtd);
    total += qtd;
}
printf("Total inserido: %d (esperado: %d)\n", total, N);
printf("Tempo: %f segundos\n", fim - inicio);

// Destruir os locks e liberar a memoria
for (int i = 0; i < num_listas; i++) {
    omp_destroy_lock(&locks[i]);
    liberar(listas[i]);
}
free(locks);
free(listas);

return 0;
}

```

4.3. OUTPUT A

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa9> ./tarefa9A
Lista A: 500154 elementos
Lista B: 499846 elementos
Total inserido: 1000000 (esperado: 1000000)
Tempo: 2.848000 segundos

```

4.4. OUTPUT B

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa9> ./tarefa9B
Lista 0: 123938 elementos
Lista 1: 124346 elementos
Lista 2: 124230 elementos
Lista 3: 124535 elementos
Lista 4: 126041 elementos
Lista 5: 125669 elementos
Lista 6: 125817 elementos
Lista 7: 125424 elementos
Total inserido: 1000000 (esperado: 1000000)
Tempo: 2.696000 segundos

```